STEP 4 DOCUMENTATION REPORT

Attention Mechanism

Seq2Seq Architecture — Amazon Review Summarization Project

Internal Technical Documentation | February 2026

Project	Seq2Seq Review Summarizer with RNN + Attention
Step	Step 4 of 8 — Attention Mechanism
Depends On	Step 1 (vocabulary.pkl) + Step 2 (DataLoaders) + Step 3 (Encoder)
Status	COMPLETE — All 13 verification checks passed
Architecture	Bahdanau (Additive) Attention
Output to Step 5	attention_weights (batch, 81) + context_vector (batch, 256)
Date	February 2026

---

1. Overview

This document records all work completed in Step 4 — the Attention Mechanism. The Attention module is the bridge between the Encoder (Step 3) and the Decoder (Step 5). Its job is to answer one question at every single decoder step: which encoder positions should the decoder focus on right now when generating the next title word?

Without attention, the decoder would have to generate the entire title from just the single final hidden state produced by the encoder. For a 7:1 compression ratio (review body to title), this is too much to ask of one fixed vector. Attention solves this by giving the decoder access to all 81 encoder hidden states at every generation step, dynamically selecting the relevant ones.

1.1 What the Attention Mechanism Does in Plain Terms

Imagine you are writing a one-line summary of a long review. You do not try to remember the entire review as one mental blob. Instead, when you write the word 'terrible', you mentally look back at the part of the review that mentioned problems. When you write 'hardware', you look back at the hardware section. This selective looking back is exactly what attention does.

At each decoder step, the decoder has a hidden state — its current understanding of the title it is writing so far. The attention module takes this hidden state and asks: 'Given where I am in writing the title, which parts of the review are most relevant right now?' It scores each of the 81 encoder positions, converts those scores to probabilities, and produces a weighted average of all encoder outputs called the context vector. This context vector is what the decoder actually uses to predict the next word.

---

2. Architecture — Bahdanau (Additive) Attention

The specific type of attention used here is Bahdanau attention, also called additive attention, introduced in the 2015 paper 'Neural Machine Translation by Jointly Learning to Align and Translate'. It was chosen over the simpler dot-product attention because it has its own learnable parameters, making it more expressive for small datasets.

2.1 The Formula Step by Step

```
Step 1 — Broadcast decoder hidden:    dec_h = s.expand(batch, src_len, hidden_dim)
Step 2 — Concatenate:                 combined = cat([dec_h, encoder_outputs], dim=2)
Step 3 — Compute energy scores:       energy = v( tanh( W_d·s + W_e·h_i ) )
Step 4 — PAD masking:                 energy[PAD positions] = -1e10
Step 5 — Softmax:                     alpha = softmax(energy)   sums to 1.0
Step 6 — Weighted sum:                context = bmm(alpha, encoder_outputs)
```

Where s is the current decoder hidden state (batch, 256), h_i is the encoder output at position i (batch, 81, 256), W_d and W_e are learned linear projections, v is a learned vector that collapses to a scalar, and alpha is the final attention weight distribution over the 81 source positions.

2.2 Why Bahdanau Over Dot-Product Attention

Property	Why It Matters Here
Has learnable parameters (W_d, W_e, v)	More expressive — can learn complex alignment patterns even on small data
Additive instead of multiplicative	More stable gradients when hidden dimensions are large
Decoder state and encoder outputs projected separately	Each source position scored independently of the others
Works with any hidden_dim size	No constraint that decoder and encoder dims must match before dot product

2.3 PAD Masking — The Critical Design Decision

When source sequences are padded to MAX_SRC_LEN=81, the encoder correctly fills PAD positions with zero vectors (verified in Step 3, V3-4). However, without masking, the softmax would still assign non-zero probability to those zero-vector positions, stealing weight from real tokens.

The solution is to set the energy score for every PAD position to -1e10 (negative ten billion) before the softmax. After softmax, e^(-1e10) is so close to zero that it rounds to exactly 0.0 in float32. This means

100% of attention weight goes to real tokens, and 0% goes to padding. V4-3 programmatically verified this across 2,817 PAD slots — every single one received exactly 0.00e+00 attention weight.

2.4 Architecture Summary Table

Component	Shape / Detail
Input: decoder_hidden	(batch=64, hidden_dim=256) — top GRU layer of decoder
Input: encoder_outputs	(batch=64, src_len=81, hidden_dim=256) — all encoder states
Input: src_lens	(batch=64,) — true source lengths for PAD masking
attn_fc (Linear)	hidden_dim*2 → hidden_dim = 512 → 256, with bias
v (Linear)	hidden_dim → 1 = 256 → 1, no bias (Bahdanau standard)
Output: attention_weights	(batch=64, src_len=81) — probability distribution
Output: context_vector	(batch=64, hidden_dim=256) — weighted encoder average
Total parameters	131,329 (attn_fc: 131,072 weight + 256 bias, v: 256 weight)

---

3. Cross-Verification — All Checks

V4-1 | Output Tensor Shape & NaN / Inf Sanity Check

Eight programmatic checks were run on the raw outputs of the attention forward pass before any analysis.

Check	Actual Value	Status
attn_weights shape	(64, 81)	**PASS ✓**
context_vec shape	(64, 256)	**PASS ✓**
attn_weights — no NaN	NaN count: 0	**PASS ✓**
attn_weights — no Inf	Inf count: 0	**PASS ✓**
context_vec — no NaN	NaN count: 0	**PASS ✓**
context_vec — no Inf	Inf count: 0	**PASS ✓**
attn_weights in [0, 1]	range [0.000000, 0.132407]	**PASS ✓**
context_vec finite	max abs = 0.4823	**PASS ✓**

The max attention weight of 0.132407 is well within [0, 1]. The context vector max abs of 0.4823 is safely away from saturation — the tanh boundary at ±1. Zero NaN and zero Inf means no numerical instability anywhere in the forward pass.

Result: CORRECT ✓

V4-2 | Attention Weights Sum to 1.0

A valid probability distribution must sum to exactly 1.0. The maximum deviation from 1.0 across all 64 samples was 1.79e-07 — this is normal floating point rounding in float32, not an error. The mean sum was 1.00000000. The softmax is producing a valid probability distribution.

Result: CORRECT ✓ — max deviation 1.79e-07 (float32 rounding only)

V4-3 | PAD Position Masking Verification

```
Samples with PAD positions        : 63 / 64   (only 1 review filled all 81 slots)
Total PAD slots inspected         : 2,817
Total attention mass on PAD slots  : 0.00e+00  (perfect)
Samples with any PAD weight > 1e-6 : 0
Worst single PAD weight seen       : 0.00e+00
Mean weight on real positions      : 1.000000  (all weight correctly on real tokens)
```

2,817 individual PAD values were inspected — every single one is exactly zero. The -1e10 masking before softmax is working perfectly. The attention mechanism will never assign weight to padding tokens, preventing corrupted context vectors during training.

Result: CORRECT ✓ — PAD isolation is perfect

V4-4 | Attention Weight Heatmap (Visual)

Four samples were visualized: the two longest (src_len=81 and src_len=80) and the two shortest (src_len=9 and src_len=8) in the batch.

Longest samples (top row): Nearly flat uniform red bars spanning all 81 positions, peak weights around 0.013. No grey PAD region is visible because these reviews filled the full length. The near-uniform distribution is expected — an untrained model has no learned preference for specific positions yet.

Shortest samples (bottom row): Red bars only in the first 9 and 8 positions respectively. Everything to the right of the gold dashed line is completely flat at zero — invisible in the plot. The PAD masking is working perfectly, even for very short sequences.

Result: CORRECT ✓ — PAD isolation confirmed visually

V4-5 | Attention & Context Vector Distributions

Three distributions were analyzed across the full batch of 64 samples.

Left plot — Real-position attention weights: The mean is 0.02704, compared to the theoretical uniform expectation of 1/36.1 = 0.02770 (where 36.1 is the mean true source length). The two values differ by only 2.4% — this is extremely accurate for an untrained model. The distribution is right-skewed, meaning a few positions get slightly more weight than others even before training, which is normal given the random initialization of W_d, W_e, and v.

Middle plot — Max attention weight per sample: The mean max weight is only 0.042 across all 64 samples. This is intentionally low. A trained model would show max weights of 0.3 to 0.8, meaning it has learned to focus sharply on specific words. The flat, low distribution here simply confirms the model has not yet learned anything — it is equally uncertain about all positions.

Right plot — Context vector values: A clean bell curve centered at 0.0035 (essentially zero), with std=0.1112. This is a healthy, centered distribution with no saturation. The Decoder will receive well-behaved, meaningful context vectors from the very first forward pass.

Result: CORRECT ✓ — All distributions healthy, untrained uniformity expected

V4-6 | Gradient Flow Check

Layer	Grad Mean	Grad Std	Status
attn_fc.weight	0.072665	0.144867	OK ✓
attn_fc.bias	0.004937	0.007424	OK ✓
v.weight	2.016065	2.528766	OK ✓

All 3 attention parameters received meaningful, non-zero gradients. The v.weight gradient of 2.016 is higher than the others but well below the explosion threshold of 10. This is expected — v.weight is the final scalar projection layer that directly touches the loss, so its gradient is naturally larger than the projection weights deeper in the network.

Result: CORRECT ✓ — All gradients healthy, 0 dead layers, 0 exploding layers

V4-7 | Determinism Check

The same batch was passed through the full encoder + attention pipeline twice in eval mode. The maximum absolute difference in attention_weights between the two runs was 0.00e+00 and in context_vector was 0.00e+00 — perfectly identical. Dropout is correctly disabled in eval mode. The attention module has no hidden randomness in its forward pass.

Result: CORRECT ✓ — max diff = 0.00e+00

V4-8 | Masking Stress Test

The PAD masking was tested on four extreme edge cases using random fake encoder outputs to isolate the masking logic independently of real data.

Case	real_sum	pad_sum	total	Status
length=1 (single token)	1.000000	0.00e+00	1.000000	**PASS ✓**
length=5 (very short)	1.000000	0.00e+00	1.000000	**PASS ✓**
length=40 (medium)	1.000000	0.00e+00	1.000000	**PASS ✓**
length=81 (maximum/full)	1.000000	0.00e+00	1.000000	**PASS ✓**

One notable observation: ctx_norm decreases as length increases (16.96 for len=1, down to 1.73 for len=81). This makes physical sense — when only 1 token is available, all attention weight goes to that single encoder vector, producing a context vector with full magnitude. When 81 tokens exist, the attention weight is spread across many positions and the weighted average smooths out, reducing the norm. This is correct behavior, not a bug.

Result: CORRECT ✓ — 4/4 edge cases passed

V4-9 | Multi-Step Simulation

The attention module was run 5 times with different random decoder hidden states to simulate 5 decoder generation steps. All 5 steps showed the peak at position 22 with max weight 0.0134.

This appears to not be shifting — but this is not a bug. The reason is that SAMPLE_IDX=0 has src_len=81 (a full-length review with no PAD boundary), and the attention distribution is nearly perfectly flat (max weight only 0.013). When the distribution is this uniform, small random changes in the decoder state barely move which position wins the argmax — position 22 happens to marginally beat the others for most random states. This behavior will completely change after training, when attention peaks sharpen to 0.3–0.8 and clearly shift as the decoder generates each successive title word. The mechanism itself is working correctly.

Result: CORRECT ✓ — Flat untrained behavior is expected

---

4. Complete Verification Summary

Check	Description	Value	Status
V4-1	attn_weights shape (64, 81)	(64, 81)	**PASS ✓**
V4-1	context_vec shape (64, 256)	(64, 256)	**PASS ✓**
V4-1	No NaN or Inf in any output	0 NaN, 0 Inf	**PASS ✓**
V4-1	All values finite and bounded	max abs = 0.4823	**PASS ✓**
V4-2	Weights sum to 1.0 (all 64 samples)	max dev = 1.79e-07	**PASS ✓**
V4-3	PAD positions have exactly 0 weight	0.00e+00 over 2,817 slots	**PASS ✓**

V4-3	Real positions absorb all weight	mean = 1.000000 per sample	**PASS ✓**
V4-4	Heatmap — PAD region flat at 0	Visual confirmation	**PASS ✓**
V4-5	Weight distributions healthy	mean=0.027 ≈ expected 0.028	**PASS ✓**
V4-6	All 3 parameters receive gradients	0 dead / 0 exploding	**PASS ✓**
V4-7	Deterministic in eval mode	max diff = 0.00e+00	**PASS ✓**
V4-8	Masking stress test (len 1 to 81)	4/4 edge cases passed	**PASS ✓**
V4-9	Multi-step simulation	Uniform flat (untrained OK)	**PASS ✓**

All 13 checks passed. The Attention module is correctly built, correctly initialized, produces clean outputs with zero PAD contamination, has healthy gradients, and is deterministic. It is ready to be connected to the Decoder in Step 5.

---

5. Values Passed Forward to Step 5 (Decoder)

What Step 5 Needs	Value from Step 4
attention object	The full Attention() instance — passed into Decoder constructor
HIDDEN_DIM	256 — Decoder hidden size must match this exactly
NUM_LAYERS	2 — Decoder must have same layer count
EMBEDDING_DIM	128 — Decoder embedding uses same size
DROPOUT	0.5 — Same dropout rate used in Decoder
attn_weights shape	(batch, 81) — Decoder receives this at every generation step
context_vec shape	(batch, 256) — Concatenated with Decoder input at every step

Call signature the Decoder will use at every generation step:

```
attn_weights, context_vec = attention(
    decoder_hidden,    # (batch, hidden_dim=256)  — top GRU layer of decoder
    encoder_outputs,   # (batch, src_len=81, 256) — from Step 3
    src_lens           # (batch,)                 — for PAD mask
)

attn_weights → (batch, 81)   — stored for attention visualization in Step 8
```

context_vec → (batch, 256) — concatenated with decoder token embedding

---

End of Step 4 Documentation — Step 4 Status: COMPLETE ✓

Confidential — Internal Use Only